第一章：绪论

Web 应用程序：是一种基于 Web 技术构建的软件应用程序，它通过浏览器与用户进行交互，并提供各种功能和服务

Internet：将全世界的上网设备通过 TCP/IP 通信协议连接到一起，无论身处何处，均可通过 Internet 获取需要的数据，它能够提供许多的服务，比如 WWW、E-Mail、FTP(文件传输协议)、Telnet(远程登录)等

万维网：是由数以千计的网站主机（服务端和客户端）通过超文本协议(HTTP)相连而成

URL 的标准格式：访问方式://完整主机域名/目录路径/网页文件文件名

Web 应用开发主要阶段：①需求分析和规划阶段、②原型设计和交互设计阶段、③技术架构设计和数据库设计阶段、④前端开发阶段/后端开发阶段/数据库开发阶段、⑤测试阶段、⑥发布和部署阶段、⑦运营和维护阶段

Web 服务器：一般指网站服务器，是指驻留于因特网上某种类型计算机的程序，可以处理浏览器等 Web 客户端的请求并返回相应响应，也可以放置网站文件，也可以放置数据文件。

Web 系统的通信过程：①用户输入网址（URL）、②域名解析器进行域名解析、③浏览器软件在网络中搜索域名所指的主机，直到找到这台主机、④通过端口，定位主机上的服务器软件、⑤服务器软件通过所给的资源名，找到对应的资源、⑥将所找的资源内容输出响应到浏览器、⑦浏览器依据获取到的资源进行渲染，展示出来

Request：浏览器向服务器发出的数据、指令或获取资源的信息；请求方式：一般是 GET 或 POST；组成：请求行(请求方法、请求地址、协议)、请求头(请求说明信息)、请求体(数据)

Response：服务器针对浏览器的请求而给出的响应内容；组成：响应行(服务器的 HTTP 版本和状态码)、响应头(响应说明信息)、响应体(数据)

第二章：HTML5

标记语言：由规定浏览器软件如何显示和管理 Web 文档的指令集组成，执行诸如显示图片、格式化文本等功能

HTML：是 WWW 的描述语言，由 TimBerners-lee 提出；是一套标记符号或代码集（用于标识结构元素、放置多媒体、或对表单进行描述）；实现了信息的平台无关性；每个标记都有特定的功能，用“<>”括起来；还可以使用属性进一步描述其功能

一个标准的 HTML5 模板：文档类型定义；用于创建一个 HTML 文档，是所有其他元素的父容器，包含 head 和 body；包含对网页文档进行描述的的信息，包含<base>,<link>,<meta>,<script>,<style>、<title>等<body>包含实际由浏览器显示的文本和元素；<!-- --> 注释标签

标记的标准语法：<标记名 属性 1=“属性值” 属性 2=“属性值”>文本或其他标记</标记名>

元数据内容标记：<base>、<link>、<meta>、<noscript>、<script>、<style>和<title>

分段/布局内容标记： 、<header>、<footer>、<article>、<aside>、<nav>、<section>

标题内容标记：<h1>、<h2>、<h3>、<h4>、<h5>、<h6>和<hgroup>

嵌入内容标记：<p>、<audio>、<canvas>、<embed>、<iframe>、<img>、<math>、<object>、<picture>、<svg>和<video>

交互内容标记：<a>、<button>、<details>、<embed>、<iframe>、<label>、<selec

t>和<textarea>

表格相关内容标记：<table>、<caption>、<thead>、<tbody>、<tr>、<td>、<th>、<tfoot>

表单相关内容标记：<button>、<fieldset>、<input>、<label>、<meter>、<object>、<output>、 <progress>、<select>和<textarea>

列表：<ol>、<ul>一般与<li>配合使用；<dl>一般与<dt>、<dd>配合使用

标记属性：class、id、lang、style、name、src、href、target

利用 HTML 标签，依据给出的图，写出表单的前端代码、<input>的 placeholder、name、type(text、button、checkbox、hidden、file、password、radio、submit、reset)

HTML5 页面设计：网站应该为目标受众设计，满足其需求，具有亲和力，友好性；常见的网站组织结构主要有：分级式组织、线性组织和随机组织；视觉原则：重复、对比、近似、对齐；

HTML5 页面设计：文本的设计原则：使用短句、言简意赅；用标题和副标题组织内容；项目列表能充分吸引人的眼球，而且易于阅读；针对不同的主题和目标用户选用合适的颜色；避免完全依赖图片来表达信息，对于表达的重点要有对应的文本描述，文件大小和图片尺寸尽量小，图片要注意抗锯齿的问题，只使用必要的多媒体提供替代文本；网站要易于导航，建议在每个页面上都提供清晰的导航链接，使其保持在页面的同一位置，并保证最大的易用性；无论用文字还是图片提供清晰的导航栏，大型网站采用面包屑路径，尽量不要用图片导航，动态导航，可以展示更多导航信息

第三章：css

层叠样式表(CSS)：是一种用来为结构化文档(如 HTML 文档或 XML 应用)添加样式(字体、间距和颜色等)的计算机语言，实现了内容和表现的分离，通常存储在样式表中，其扩展名为.css

CSS 实现方式：行内样式(style=“ ”)、内部样式(<style></style>)、外部样式(<linkref=“stylesheet”href=“css/*.css”>)，就近原则

基本语法：选择器{属性：值;[属性：值;]}，注释：/**/多行或单行

选择器分类：基本选择器(全局选择器-*标签选择器-标签名、类选择器-.class 名和 id 选择器-#id 名)；

层次选择器(后代选择器-父标签后代标签、子选择器-父标签>子代标签、相邻兄弟选择器-被标记标签+兄弟标签、通用兄弟选择器-被标记标签~兄弟标签)；伪类选择器(结构性-:结构描述符、状态性-:状态描述符)；属性选择器(属性选择器-标记[属性名]、属性和值选择器-标记[属性名多种等式属性值])

字体：color 颜色、font-family 字体名称、font-size 字体大小、font-style 字体斜体、font-weight 字体粗细、font 字体设置

段落：text-align 文本对齐、text-decoration 文本装饰、text-transform 文本转换、text-indent 文本缩进

背景：background-color 背景颜色、background-image 背景图片、background 背景……

列表：list-style-type 列表样式、list-style-image 列表图片样式、list-style 列表样式……

表格：border 边框、width/hight 宽/高、text-align 文本对齐……

盒子模型：HTML 元素都可看作盒子，包含外边距、边框、内边距和内容

边框：border-style 边框样式、border-width 边框宽度、border-color 边框颜色、border 边框设置

轮廓：outline-style 轮廓样式、outline-width 轮廓宽度、outline-color 轮廓颜色、outli

ne 轮廓设置

其他基础设置：margin 外边距宽、padding 填充宽、display 显示、visibility 可见性、float 浮动、clear 清除浮动、opacity 不透明度、overflow 溢出

高级设置：filter 滤镜、transform 转换、transition 过渡效果、animation 动画

第四章：JS

JavaScript：是一种描述性、基于对象和事件驱动的、具有安全性能的轻量级脚本语言；可实现客户端表单验证(数据处理)，制作页面动态特效(元素交互)等功能；可广泛用于服务器、PC、笔记本电脑、平板电脑和智能手机等设备；由 ECMAScript、BOM 和 DOM 组成

基本特点：简单性：语言控制结构和变量类型简单；安全性：数据仅通过浏览器展示和交互，避免丢失；动态性：直接对用户输入做出响应；跨平台性：依赖浏览器运行，与操作环境无关

基本用法：HTML 中的脚本必须位于<script>与</script>标签之间；脚本可被放置在 HTML 页面的<body>和<head>部分中；也可以保存为外部文件，以“*.js”命名；用<script>标签的 src 属性设置文件引用；直接写在 HTML 标签中；一般会在某事件发生后执行相应代码，包括页面加载、鼠标事件、键盘事件等

基本语法：字面量：数字(Number)/字符串(String)/数组(Array)/对象(Object)/函数(Function)；变量：声明 var、作用域(全局变量、局部变量、未声明默认为全局)、优先级；语句：一条语句以分号“;”结尾、按照编写顺序依次执行每条语句、代码块用{}括起来，大小写敏感；运算符(算术、赋值、字符串、比较、逻辑、条件)、注释(单行//，多行/**/)数据类型：JavaScript 支持字符串(String)、数字(Number)、布尔(Boolean)、数组(Array)、对象(Object)、空(Null)、未定义(Undefined)；函数、条件、循环

浏览器对象模型(BrowserObjectModel)：是 JavaScript 的组件之一，用来控制浏览器的各种操作，BOM 提供了独立于内容、可以与浏览器窗口交互一系列对象，分别对浏览器进行不同的交互操作

window：浏览器打开一个 HTML 文档时，就会创建一个 window 对象；closed/status/innerWidth/innerHeight、prompt()/alert()/confirm()/close()/open()/moveTo(x,y)/focus()

document：指每个载入到浏览器窗口中的 HTML 文件；body/cookie/URL/domain/all/forms/images/open()/write()/close()/getElementById()/getElementByName()/getElementByTagName()/getElementByClassName()

location：当前页面的 URL 地址的各种信息，如协议、主机服务器和端口号等，protocol/host/hostname/port/pathname/href/hash/search，assign(url)/reload(foece)/replace(url)

history：当前窗口所访问过的 URL 地址，length，back()/forward()/go(n)

screen：客户端显示屏幕的信息，width/height/availWidth/availHeight/colorDepth

navigator：浏览器的相关信息，appCodeName/appName/appVersion/onLine/platform/userAgent

文档对象模型(DocumentObjectModel)，是一个和 JavaScript 进行内容交互的 API，为文档提供了一个层次化的节点树，操作 DOM 可修改文档的内容和展现形式，实现页面的各种效果

DOM 主要功能：改变页面的所有 HTML 元素、改变页面的所有 HTML 属性、改变页面的所有 CSS 样式、对页面的所有事件做出反应

DOM 节点及节点间的关系：节点(文档节点、元素节点、文本节点、属性节点、注释节点)，关系(根、父节点、子节点、同胞节点)

节点访问：getElementById(str)/getElementByName(str)/getElementByTagName(str)get

ElementByClassName(str)、parentNode/children/firstChild/lastChild/firstElementChild/lastElementChild/nextElementSibling、previousElementSibling

节点操作：getAttribute()/setAttribute()、createElement(tagName)/A.appendChild(B)/insertBefore(A,B)/cloneNode(deep)、removeChild(node)/replaceChild(newNode,oldNode)

修改节点样式：修改节点 style 属性、设置 class 属性、使用 style 对象的 CSSText 属性、使用 setAttribute 方法

jQuery：是一个轻量级、免费、开源的 JavaScript 库；能实现 HTML 元素选取、操作、CSS 操作、HTML 事件函数、DOM 遍历和修改、编辑特效和动画、AJAX 等功能；兼容所有主流浏览器；支持链式语法；

安装：从 CDN(内容分发网络)中载入、下载然后<script>标签的 src 引入

基本语法：$(selector).action()

Bootstrap：是一套现成的 CSS 样式集合，是一个集成了 HTML、CSS 和 JS 的前端框架可用于开发响应式布局、移动设备优先的 WEB 项目，由 Twitter 工程师为了方便内部工作而设计开发，并开源；适合那些没有专业设计团队的项目开发

安装：从 CDN(内容分发网络)中载入、下载然后<script>的 src 引入，<link>标签的 href 引入

第五章：flask 安装及其基本结构

安装：创建应用目录、配置虚拟环境、安装依赖包

基本结构：导入 Flask、新建应用实例、创建路由、运行程序

定义路由：@app.route()、add_url_rule(url,endpoint,view_function,methods)

视图函数：request、response(字符串/html、jsonify、Response/make_response()、render_template())

程序运行：app.run(debug，host，port)

Hello World 的代码：

```
from flask import Flask
app = Flask (_ _name_ _)
@app.route ('/')
def hello():
    return 'Hello World!'
if _ _name_ _== '_ _main_ _':
    app.run(debug = True)
```

第六章：Flask-模板

模板的作用：实现业务逻辑和表现逻辑分离，方便分工、提高应用的可维护性

模板的主要内容：模板引擎、模板渲染、模板变量、模板过滤器、模板控制结构、模板继承

Jinja2：是一个现代的，设计者友好的，仿照 Django 模板的 Python 模板语言。它速度快，被广泛使用，并且提供了可选的沙箱模板执行环境保证安全；Flask 使用 Jinja2 作为默认模板引擎。可以使用其它模板引擎。但是不管使用哪种模板引擎，都必须安装 Jinja2。因为使用 Jinja2 可以让 Flask 使用更多依赖于这个模板引擎的扩展

模板渲染：flask 使用 render_template(模板名，key=value，key=value)完成模板渲染

模板变量：{{变量}}，支持所有类型的变量，变量在模板中用 key 占位，渲染时由 value 替换 key

过滤器：{{变量名|过滤器名}}，实现对变量的处理、可传递参数/自定义、可链式调用

控制结构：{%if%}{%endif%}，{%foriteminitems%}{%endfor%}

继承：父模板中用{%block 块名%}{%endblock%}定义变化的部分，子模块中用{%extends“父模块”%}继承父模块，用{%block 块名%}{%endblock%}在其内部写上子模块自己的内容

Include，宏，url_for()函数

第七章 Flask-Web 表单

Flask-WTF：WTForms(为 pythonWeb 开发服务的一个灵活的表单验证、渲染库，能够用于多种框架和模板引擎，支持数据验证，CSRF(跨站请求伪造)保护)进行了封装，方便集成到 Flask 应用中，可实现在后端定义表单项，利用参数传递到模板中，来实现表单创建，使用 pip install flask-wtf 来安装

实现一个基于 Flask-WTF 的 Web 表单的基本流程：配置、构建表单类、创建表单实例、表单渲染成 HTML、提交验证

配置：不用在应用层创建实例，只需要配置一个随机字符串作为密钥，app.config['SECRECT_KEY']

构建表单类：服务器端每个 Web 表单都由一个继承自 FlaskForm 的类，class UserForm(FlaskForm):字段名=字段对象(标签名,表单验证项(validators))；StringField，PasswordField，SubmitField，TextAreaField，IntegerField，RadioField，SelectField，HiddenField；DataRequired,Length，EqualTo，NumberRange

创建表单实例：uform=UserForm()

表单渲染成 HTML：render_template('user.html',uform=uform)；{{uform.password(id="myPassWord")}}

提交验证：uform.validate_on_submit()，可以自定义验证函数 validate_<fieldname>(self,data)

第八章：Flask 数据库

ORM：是一种程序技术，用于实现面向对象编程语言里不同类型系统的数据之间的转换；提供了概念性的、易于理解的模型化数据的方法；是在数据库引擎之上又进行了一层封装，对于数据的操作，无需再去编写原生 sql，取而代之的是基于面向对象的思想去编写类、对象、调用相应的方法等，ORM 会将其转换/映射成原生 SQL 然后交给数据库引擎来执行；类对应一张表；属性对应字段；一个类的实例对象对应表中一条数据；ORM 帮我们做了 SQL 优化且支持多种数据库，这极大地提升了我们的开发效率

Flask-SQLAlchemy：是一个 Flask 扩展，简化了在 Flask 应用使用 SQLAlchemy 的操作；是一个强大的关系型数据库框架，支持多种数据库后台；提供了高层 ORM，也提供了使用数据库原生 SQL 的低层功能；目前 python 中最强大的 ORM 框架，功能全面，使用简单；使用 pip install flask-sqlalchemy 安装

实现一个基于 Flask-SQLAlchemy 的数据操作基本流程：导入与配置、初始化、构建模型类、数据库基本操作

导入与配置： from flask_sqlalchemy import SQLAlchemy；构建一个空数据库，设置数据库配置 app.config['SQLALCHEMY_DATABASE_URI'] = "mysql+pymysql://root:123456@127.0.0.1:3306/flask_demo“

初始化：db = SQLAlchemy(app)， db = SQLAlchemy() /db.init_app(app)

构建模型类：模型类必须继承 db.Model，表名默认为类名小写，可以通过 __tablename__类属性进行修改；类属性对应字段，必须是通过 db.Column() 创建的对象；可以通过 c

reate_all() 和 drop_all()方法来创建和删除所有模型类对应的表；String、Integer、Text…；primary_key、unique、index、nullable…

数据库基本操作：需在 with app.app_context():语句下执行数据库基本操作；插入行（新建表格类的实例对象 role_admin = Role(name = 'Admin')，将对象添加进数据库会话 db.session.add(role_admin)，执行会话提交 db.session.commit())；修改行（类实例中修改相关属性 role_admin.name = 'Administrator'将修改后的对象添加进数据库会话 db.session.add(role_admin)执行会话提交 db.session.commit())；删除行（会话中删除实例 db.session.delete(role_mod)，执行会话提交 db.session.commit()；查询行（使用 query 对象 Role.query.all())

关系构建：一对多：在多端创建一个外键属性，能将其与一端的 id 建立连接 role_id = db.Column(db.Integer,db.ForeignKey('role.id'))；在一端用 relationship 定义一个多端的列表，用于查询一端中有多少多端的实例；这种作用主要体现在 View 和 Template 中，不会体现在数据库内 users=db.relationship('User', backref='role')；多对多：使用第三张表完成多对多的关联关系，将多对多的关系分解为原表和关联表之间的两个一对多的关系，使用定义一对多关系对多对多进行编码， relationship 端必须把 secondary 参数设为关联表

查询：Flask-SQLAlchemy 为每个模型类都提供了 query 对象；查询所有：模型类.query.all()；条件查询：filter_by(字段名 = 值)； get(n)； filter(模型类.字段==值)； all()/first()、endswith('z')/startswith('z')/ contains('z')/like('z%')、 or_/and_/not_/ <、>、__lt__、__gt__ __le__、__ge__ /in_/between；排序：order_by(字段)；限定数量： offset(数量)/limit(数量)

迁移的必要性：在应用开发的过程中，不可避免会修改数据库模型，就需要同步更新数据库表；但是 flask-SQLAlchemy，只有在数据库中表不存在时，才会依据模型创建新表，如此为了把我修改后的模型进行更新的方式就只有先删除数据库中原有的表，然后再新建表；这样就会丢失原数据表中的数据，如何才能又能保存原数，又能更新模型，最好的方式就是用数据库迁移框架来实现

使用 Flask-Migrate 数据库迁移实现流程：安装 pip install flask-migrate，创建迁移仓库（项目启动文件中导入：from flask_migrate import Migrate、项目启动文件中实例化：migrate = Migrate(app,db)、终端命令行中 执行：flask db init)；创建迁移脚本（终端命令行中执行： flask db migrate [–m "修改说明"]，在 version 文件夹下生成迁移脚本 xxx.py)；更新数据库（终端命令行执行： flask db upgrade，实现模型改动向数据库的迁移，终端命令行执行： flask db downgrade，依据迁移文件回退到数据库上一次修改的地方）

第九章：大型应用的结构及蓝图

结构化系统文件的必要性：对于小型的 WEB 应用，使用单一的脚本文件方便、快捷，阅读方便，但是其伸缩性不好，当应用的功能变多、应用变复杂后，使用单个大型源码文件会导致很多问题，主要体现在可维护性上；有必要对大型应用采用相应的结构化文件组织，方便后期维护管理

一般的组织结构：启动文件命名为 app.py 置于项目根目录下，app.run()；配置文件命名为 settings.py/ config.py 置于项目根目录下， app.config.from_object() ；扩展实例置于 exts 包下的_ _init_ _.py 中，exts 包置于项目根目录下；工厂函数置于 apps 包的_ _init_ _.py 中， create_app() ， apps 包置于项目根目录下；各模块内容，按照模块分别创建文件夹置于 apps 包，模块文件夹内分别将各模块的模型(model.py)、视图(view.py)、表单(forms.py)等相关内容； 模板文件置于 templates 中，将各模块的模板分别放置在以模块命名的文件夹内；静态文件按照 css、js、image 创建文件夹，置于 static 文件夹下

蓝图实现流程：在模块的_ _init_ _.py 中，导入蓝图模块，并实例化一个蓝图对象，同

时将需要用到蓝图的视图导入_ _init_ _.py，在使用该蓝图的视图文件中引入实例化的蓝图，from flask import Blueprint，user_bp = Blueprint('user',_ _name_ _)，from.import views，from.import user_bp；在视图模块中使用与应用类似的方式，即装饰器@user_bp.route()或add_url_rule()函数定义路由；在工厂函数中将蓝图注册到应用上 app.register_blueprint(main_bp）；利用 url_for()反向构建路由的 URL，{{url_for('user_bp.detail')}}

第十一章 REST、其他拓展、测试及部署：

REST：一种富互联网应用（RIA）的架构；具有客户端–服务器、无状态、缓存、接口统一、系统分层、按需编程的特性；请求方式：GET(获取)、POST(创建)、PUT(修改)、DELETE(删除)；JOSN

REST 实现：使用 Flask 创建 REST 式 Web 服务十分简单。使用熟悉的 route()装饰器及其 methods 可选参数可以声明服务所提供资源 URL 的路由；处理 JSON 数据同样简单，请求中的 JSON 数据可以通过 request.get_json()转换成字典格式，而且可以使用 Flask 提供的辅助函数 jsonify()，从 Python 字典中生成需要包含 JSON 的响应

其他扩展：下载安装、实例化和初始化、扩展使用；Flask-Mail、Flask-Login、Flask-Moment、其他

测试：使用技术手段验证软件是否满足使用需求，其目的在于减少软件缺陷（BUG），保障软件质量；Web 应用的测试内容：功能、性能、用户界面、兼容性、安全、接口…；Unittest、Pytest

部署：流程：架设服务器、配置、代码同步、开启服务器、运行服务器、访问；自带服务器、稳定服务器 uWSGI、Heroku 平台、Docker 容器